```
from google.colab import files

uploaded = files.upload()
```

Chọn tệp UD_English-EWT.tar.gz
UD_English-EWT.tar.gz(application/x-gzip) - 3729474 bytes, last modified: 16/10/2025 - 100% done
Saving UD_English-EWT.tar.gz to UD_English-EWT.tar.gz

```
import tarfile

tar_path = "/content/UD_English-EWT.tar.gz"

with tarfile.open(tar_path, "r:gz") as tar:
    tar.extractall("/content/")
```

```
/tmp/ipython-input-3084853600.py:6: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and reject f
  tar.extractall("/content/")
```

```
# ===== Task 1: Load .conllu và build vocab =====
import os
from collections import Counter

def load_conllu(file_path):
    sentences = []
    cur_words = []
    cur_tags = []
    with open(file_path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                if cur_words:
                    sentences.append(list(zip(cur_words, cur_tags)))
                    cur_words, cur_tags = [], []
                continue
            if line.startswith("#"):
                continue
            parts = line.split("\t")
            if len(parts) < 4:
                continue
            idx = parts[0]
            if "-" in idx or "." in idx:
                continue
            word = parts[1]
            upos = parts[3]
            cur_words.append(word)
            cur_tags.append(upos)
    if cur_words:
        sentences.append(list(zip(cur_words, cur_tags)))
    return sentences

def build_vocabs(train_sentences, min_freq=1):
    counter = Counter()
    tag_set = set()
    for sent in train_sentences:
        for w, t in sent:
            counter[w] += 1
            tag_set.add(t)
    words = [w for w, c in counter.items() if c >= min_freq]
    word_to_ix = {"<PAD>": 0, "<UNK>": 1}
    for i, w in enumerate(words, start=2):
        word_to_ix[w] = i
    tag_to_ix = {tag: i for i, tag in enumerate(sorted(tag_set))}
    return word_to_ix, tag_to_ix

data_dir = "/content/UD_English-EWT"
train_file = os.path.join(data_dir, "en_ewt-ud-train.conllu")
dev_file   = os.path.join(data_dir, "en_ewt-ud-dev.conllu")

# Kiểm tra file tồn tại
assert os.path.exists(train_file), f"Không tìm thấy train file: {train_file}"
assert os.path.exists(dev_file), f"Không tìm thấy dev file: {dev_file}"

# Load dữ liệu
```

```
train_sents = load_conllu(train_file)
dev_sents   = load_conllu(dev_file)

print("Số câu (train):", len(train_sents))
print("Số câu (dev):", len(dev_sents))
print()

# In 3 câu mẫu đầu (word, tag)
print("Ví dụ 3 câu đầu (word, UPOS):")
for i, s in enumerate(train_sents[:3], 1):
    print(f"Sentence {i}: {s[:20]}")

# Build vocabs
word_to_ix, tag_to_ix = build_vocabs(train_sents, min_freq=1)
print("\nKích thước từ vựng (word_to_ix):", len(word_to_ix))
print("Kích thước nhãn (tag_to_ix):", len(tag_to_ix))
print("\nMột vài nhãn (tag_to_ix) sample:", list(tag_to_ix.items())[:10])
```

```
Số câu (train): 12544
Số câu (dev): 2001

Ví dụ 3 câu đầu (word, UPOS):
Sentence 1: [('Al', 'PROPN'), ('-', 'PUNCT'), ('Zaman', 'PROPN'), (':', 'PUNCT'), ('American', 'ADJ'), ('forces', 'NOUN'), ('kil
Sentence 2: [('[', 'PUNCT'), ('This', 'DET'), ('killing', 'NOUN'), ('of', 'ADP'), ('a', 'DET'), ('respected', 'ADJ'), ('cleric',
Sentence 3: [('DPA', 'PROPN'), (':', 'PUNCT'), ('Iraqi', 'ADJ'), ('authorities', 'NOUN'), ('announced', 'VERB'), ('that', 'SCONJ

Kích thước từ vựng (word_to_ix): 19675
Kích thước nhãn (tag_to_ix): 17

Một vài nhãn (tag_to_ix) sample: [('ADJ', 0), ('ADP', 1), ('ADV', 2), ('AUX', 3), ('CCONJ', 4), ('DET', 5), ('INTJ', 6), ('NOUN'
```

```
# ===== Task 2: Dataset, collate_fn và DataLoader =====
import torch
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

PAD_IDX = word_to_ix["<PAD>"]
PAD_TAG = -100

class POSDataset(Dataset):
    def __init__(self, sentences, word_to_ix, tag_to_ix):
        self.sentences = sentences
        self.word_to_ix = word_to_ix
        self.tag_to_ix = tag_to_ix

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        sent = self.sentences[idx]

        words = [w for w, t in sent]
        tags = [t for w, t in sent]

        word_ids = [self.word_to_ix.get(w, self.word_to_ix["<UNK>"]) for w in words]
        tag_ids = [self.tag_to_ix[t] for t in tags]

        return torch.tensor(word_ids, dtype=torch.long), \
               torch.tensor(tag_ids, dtype=torch.long)

def collate_fn(batch):
    """
    batch = list of (word_tensor, tag_tensor)
    ta cần pad cả 2 theo độ dài lớn nhất trong batch.
    """
    word_seqs = [item[0] for item in batch]
    tag_seqs = [item[1] for item in batch]

    padded_words = pad_sequence(word_seqs, batch_first=True, padding_value=PAD_IDX)

    padded_tags = pad_sequence(tag_seqs, batch_first=True, padding_value=PAD_TAG)

    return padded_words, padded_tags

# Tạo dataset
train_dataset = POSDataset(train_sents, word_to_ix, tag_to_ix)
```

```
dev_dataset   = POSDataset(dev_sents,  word_to_ix, tag_to_ix)

# DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    collate_fn=collate_fn
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=32,
    shuffle=False,
    collate_fn=collate_fn
)

print("Train batches:", len(train_loader))
print("Dev batches:", len(dev_loader))

# Kiểm tra 1 batch mẫu
for X_batch, y_batch in train_loader:
    print("Words:", X_batch.shape)
    print("Tags :", y_batch.shape)
    break
```

```
Train batches: 392
Dev batches: 63
Words: torch.Size([32, 68])
Tags : torch.Size([32, 68])
```

```
# ===== Task 3: Xây dựng mô hình RNN =====
import torch
import torch.nn as nn

class SimpleRNNForTokenClassification(nn.Module):
    def __init__(self, vocab_size, tagset_size, embed_dim=128, hidden_dim=128):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=word_to_ix["<PAD>"])

        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        self.fc = nn.Linear(hidden_dim, tagset_size)

    def forward(self, x):
        """
        x: (batch_size, seq_len)
        """
        emb = self.embedding(x)
        rnn_out, _ = self.rnn(emb)
        logits = self.fc(rnn_out)
        return logits

# Khởi tạo mô hình
vocab_size = len(word_to_ix)
num_tags = len(tag_to_ix)

model = SimpleRNNForTokenClassification(
    vocab_size=vocab_size,
    tagset_size=num_tags,
    embed_dim=128,
    hidden_dim=128
)

print(model)
```

```
SimpleRNNForTokenClassification(
  (embedding): Embedding(19675, 128, padding_idx=0)
  (rnn): RNN(128, 128, batch_first=True)
  (fc): Linear(in_features=128, out_features=17, bias=True)
```

```
)
```

```
# ===== Task 4: Huấn luyện mô hình =====
import torch.optim as optim

# Chọn device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Loss function & optimizer
criterion = nn.CrossEntropyLoss(ignore_index=PAD_TAG)
optimizer = optim.Adam(model.parameters(), lr=0.001)

num_epochs = 5

for epoch in range(1, num_epochs + 1):
    model.train()
    total_loss = 0
    for X_batch, y_batch in train_loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch)
        outputs = outputs.view(-1, num_tags)
        y_batch_flat = y_batch.view(-1)

        loss = criterion(outputs, y_batch_flat)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
    avg_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch}/{num_epochs}, Loss: {avg_loss:.4f}")
```

```
Epoch 1/5, Loss: 1.0502
Epoch 2/5, Loss: 0.5751
Epoch 3/5, Loss: 0.4270
Epoch 4/5, Loss: 0.3358
Epoch 5/5, Loss: 0.2704
```

```
# ===== Task 5: Đánh giá mô hình =====
def evaluate(model, data_loader):
    model.eval()
    total_correct = 0
    total_tokens = 0

    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            outputs = model(X_batch)
            preds = torch.argmax(outputs, dim=-1)

            mask = y_batch != PAD_TAG
            total_correct += (preds == y_batch).masked_select(mask).sum().item()
            total_tokens += mask.sum().item()
    return total_correct / total_tokens

# Accuracy train/dev
train_acc = evaluate(model, train_loader)
dev_acc = evaluate(model, dev_loader)
print(f"Train Accuracy: {train_acc:.4f}")
print(f"Dev Accuracy  : {dev_acc:.4f}")
```

```
Train Accuracy: 0.9305
Dev Accuracy  : 0.8708
```